Reinforcement Learning for Motion Control in Embodied AI: A Complete Tutorial

Course home · 简体中文 · PDF

Item	Details
Applicability	Isaac Lab / RSL-RL workflows are illustrative; task, trainer, asset and actuator versions must be bound together
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

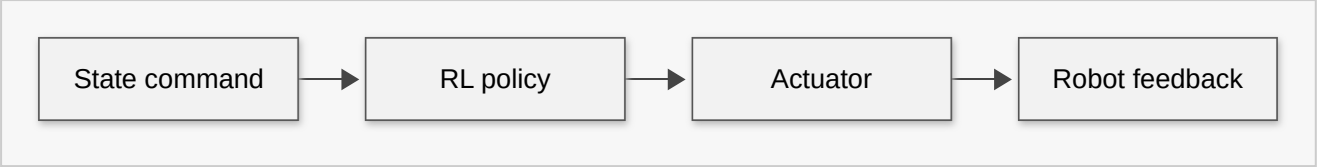
Contents

- Chapter 1: What Is Motion-Control Reinforcement Learning?
  - 1.1 Where motion-control RL fits in an embodied system
  - 1.2 Motion-control RL versus RL post-training of embodied foundation models
- Chapter 2: A Taxonomy of Motion-Control RL
  - 2.1 Classification by robotic task
  - 2.2 Classification by training strategy
- Chapter 3: Algorithms for Motion-Control RL
  - 3.1 PPO: a standard baseline for large-scale simulation
  - 3.2 SAC: improved reuse of interaction data
  - 3.3 PPO versus SAC
- Chapter 4: Designing a Motion-Control RL Environment
  - 4.1 Observation: what can the robot sense?
  - 4.2 Action: what should RL output?
  - 4.3 Reward: what behavior should the robot optimize?
  - 4.4 Reset: which initial states should be sampled?
- Chapter 5: Training a Complete Motion Controller
  - 5.1 The standard workflow
  - 5.2 Step 1: build a robot simulation model
  - 5.3 Step 2: choose control frequencies
  - 5.4 Step 3: parallel training
- Chapter 6: Motion-Imitation Reinforcement Learning

    - 6.1 The basic pipeline
    - 6.2 Motion-tracking rewards
    - 6.3 DeepMimic
    - 6.4 AMP: Adversarial Motion Priors
    - 6.5 Motion tracking versus AMP
    - 6.6 From tracking one clip to universal control
- Chapter 7: Teacher–Student Reinforcement Learning
    - 7.1 Why use teacher–student training?
    - 7.2 Standard training procedure
    - 7.3 Asymmetric actor–critic versus teacher–student
    - 7.4 RMA: Rapid Motor Adaptation
- Chapter 8: Case Study I — Training Unitree G1 Walking with PPO
    - 8.1 Task definition
    - 8.2 Observation
    - 8.3 Action
    - 8.4 Reward components
    - 8.5 Curriculum learning
    - 8.6 Open-source starting point
- Chapter 9: Case Study II — Arm and Dexterous-Hand RL
    - 9.1 Task definition
    - 9.2 Recommended control architecture
    - 9.3 Three training routes
    - 9.4 Observation design
    - 9.5 Reward design
    - 9.6 Proposed training stages
- Chapter 10: Simulation-to-Real Transfer
    - 10.1 Why simulation success does not ensure hardware success
    - 10.2 Domain randomization
    - 10.3 Sim-to-sim evaluation
    - 10.4 Real-robot deployment checklist
- Chapter 11: Evaluating Motion-Control RL
    - 11.1 Legged-robot metrics
    - 11.2 Motion-tracking metrics
    - 11.3 Dexterous-manipulation metrics
- Chapter 12: Open-Source Frameworks and Tools
    - 12.1 Isaac Lab
    - 12.2 RSL-RL
    - 12.3 MuJoCo Playground
    - 12.4 Which framework should you choose?
- Chapter 13: Representative Papers and Reading Order

- Chapter 14: Learning and Research Roadmap
  - Stage 1: basic motion-control RL
  - Stage 2: motion-tracking RL
  - Stage 3: teacher–student and sim-to-real
  - Stage 4: complex motion control
  - Stage 5: universal motion controllers
- Chapter 15: Summary and Main Takeaways
- References and Open-Source Links

Chapter concept map



Chapter 1: What Is Motion-Control Reinforcement Learning?

1.1 Where motion-control RL fits in an embodied system

A robotic system can be approximately organized into three levels:

Level	Main responsibility	Common techniques
High level: task planning	Decide what the robot should do	LLMs, VLMs, VLAs, task planning
Middle level: motion and skill control	Decide how the robot should move	Motion planning, motion tracking, RL
Low level: actuator control	Execute motor-level commands	PID, PD, impedance and current control

Motion-control RL is mostly applied at the middle level, although it may also learn parts of low-level control. For example, a high-level system may request “walk forward at 0.5 m/s.” The motion policy produces joint commands, and a low-level controller executes them.

```
User: "Walk to the table and pick up the cup"
                    |
                    v
             VLM / VLA / Planner
                    |
                    v
            Velocity / Pose Goal
                    |
                    v
             RL Motion Policy <--- Sensor feedback
                    |
                    v
            Joint / Cartesian Command
                    |
                    v
            PD / IK / Motor Driver
                    |
```

```
                    v
                Real Robot
```

A goal-conditioned motion policy can be written as

$$a_t = \pi_\theta(o_t, c_t),$$

where o_t contains observations such as joint states, IMU readings, and vision; c_t is a desired velocity, pose, or skill command; a_t is a joint or Cartesian action; and π_θ is the learned policy.

Conventional controllers typically employ explicitly designed control laws, whereas motion-control RL learns a feedback policy from interactions with a dynamical system. These approaches are complementary: practical robotic systems often combine RL with PD, inverse kinematics (IK), model-predictive control (MPC), and whole-body control (WBC).

1.2 Motion-control RL versus RL post-training of embodied foundation models

Dimension	Motion-control RL	RL post-training for foundation policies
Primary objective	Learn stable, flexible motor skills	Improve an existing policy's task performance
Typical inputs	Joints, IMU, goals, other robot states	Images, language, robot states
Typical outputs	Joint or end-effector control commands	Action chunks or action distributions
Typical models	MLPs, GRUs, Transformers	VLAs, diffusion policies, flow-matching models
Methods	PPO, SAC, imitation-based RL	PPO, SAC, offline RL, specialized fine-tuning
Main interaction source	Large-scale simulation	Pretraining data, simulation, real interaction
Typical tasks	Walking, jumping, stairs, dexterity	Visuolinguistic manipulation tasks
Main research issues	Stability, robustness, motor performance	Generalization, sample efficiency, adaptation

For instance, training a Unitree G1 to walk under velocity commands is motion-control RL. Improving the grasping success rate of an already pretrained $\pi_{0.5}$ policy through RL is foundation-policy post-training. The boundary is not absolute: both may optimize the same robot actions for manipulation, but differ in architecture, initialization, and objectives.

Chapter 2: A Taxonomy of Motion-Control RL

2.1 Classification by robotic task

Category	Typical tasks	Main challenges
Legged locomotion	Walking, running, turning	Balance and contact stability
Agile locomotion	Jumping, stair climbing, obstacles, rolls	Dynamic contact and nonlinear dynamics
Whole-body motion tracking	Dance, motion imitation, trajectory tracking	Whole-body coordination and fidelity
Manipulation control	Grasping, carrying, insertion	Contacts, accuracy, force control

Category	Typical tasks	Main challenges
Dexterous manipulation	In-hand rotation, regrasping, tool use	High-dimensional actions, multi-contact dynamics
Loco-manipulation	Walking while grasping, carrying, door opening	Coordination between locomotion and manipulation

These categories overlap. A humanoid carrying a box may simultaneously require locomotion, whole-body coordination, and manipulation.

A. Legged locomotion

The objective is to move stably under a command. A common velocity command is

$$c_t = (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, \omega_z^{\mathrm{cmd}}),$$

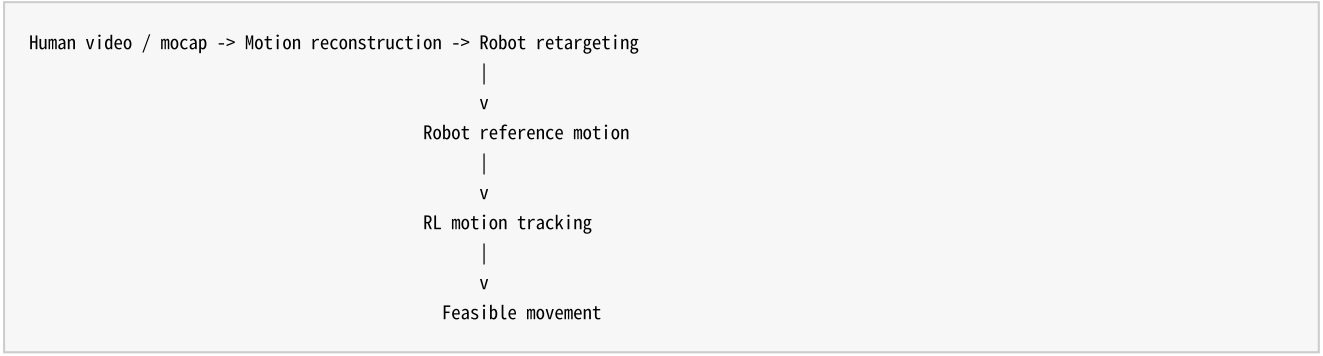
representing longitudinal, lateral, and yaw velocities. The policy outputs joint actions that track these commands. Quadruped and bipedal locomotion have developed relatively mature large-scale simulation workflows. A representative example is *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning*, which trained and deployed an ANYmal controller using massively parallel simulation [1].

B. Agile locomotion

Agile locomotion includes jumping, stair climbing, obstacle crossing, running, fall recovery, and mid-air posture control. Compared with basic walking, the robot must exploit its dynamics and contact transitions. Climbing stairs, for example, requires coordinated foot clearance, landing timing, and center-of-mass transfer.

C. Whole-body motion tracking

The robot is given a reference motion $q^{\mathrm{ref}}(t)$ and learns to track it while remaining dynamically feasible.



Representative approaches include DeepMimic, AMP, HOVER, and BeyondMimic. They explore reference imitation, motion-prior learning, multimodal whole-body control, and difficult humanoid motions [2][3][4].

D. Manipulation control

The goal is to pick up, carry, insert, or otherwise manipulate objects. RL can output joint commands or end-effector pose increments; an IK, operational-space control (OSC), or joint controller may then convert these into actuator commands. Compared with locomotion, these tasks emphasize end-effector accuracy, interaction forces, task success, collision safety, and recovery.

E. Dexterous manipulation

A multi-fingered hand controls several contacts simultaneously:

$$a_t = [a_t^{\text{finger1}}, a_t^{\text{finger2}}, \ldots, a_t^{\text{fingerN}}]^\top.$$

Tasks include stable grasps, in-hand rotation, regrasping, tool use, and recovery after object slip. The primary difficulties are high-dimensional actions and complex finger–object contact dynamics.

F. Loco-manipulation

Loco-manipulation combines moving the robot and interacting with objects. Examples include carrying while walking, using a hand for support during obstacle crossing, manipulating objects with both hands while moving, and opening a door. The objective shifts from isolated skills to coordinated whole-body behavior and contact planning.

2.2 Classification by training strategy

Strategy	Main idea	Suitable applications
RL from scratch	Explore directly with task rewards	Basic balance and walking
Imitation RL	Use rewards defined by reference motions	Dance and motion reproduction
Residual RL	Learn corrections to a baseline controller	Trajectory repair and contact compensation
Hierarchical RL	High-level skill selection and low-level control	Multistage tasks
Teacher–student learning	Train a privileged teacher and distill it	Vision-based control and sim-to-real
Adaptive RL	Adapt through historical observations	Changing friction, payloads, dynamics

These strategies can be combined. For instance, train a privileged teacher with imitation-guided RL, distill its behavior into a student that only sees deployable sensors, and transfer that student to hardware.

Chapter 3: Algorithms for Motion-Control RL

3.1 PPO: a standard baseline for large-scale simulation

Proximal Policy Optimization (PPO) is an on-policy RL algorithm widely used for quadrupeds, bipeds, humanoid motion tracking, and dexterous manipulation. A typical loop is

```
Initialize Actor / Critic
          |
          v
Create thousands of parallel environments
          |
          v
Actor collects (Observation, Action, Reward)
          |
          v
Compute Advantages
          |
          v
Update Actor and Critic -> Repeat
```

PPO is particularly effective when coupled with GPU-parallel physics simulation. Although it reuses older data only to a limited extent, thousands of simulated robots can generate large batches of fresh interactions. For instance, 4,096 environments can simulate independent copies of the same robot [1].

A widely used implementation is RSL-RL, which provides PPO, neural-network components, and training utilities aimed at robotic control [5].

3.2 SAC: improved reuse of interaction data

Soft Actor-Critic (SAC) is an off-policy algorithm. A replay buffer stores past transitions so that they can contribute to multiple updates.

```
Robot / Simulator
        |
        v
   Experience
        |
        v
   Replay Buffer
     /      \
    v        v
Critic      Actor
Update      Update
     \      /
       New Policy
```

Robot interaction may be expensive in the real world, making SAC's data reuse attractive. SAC is a useful candidate for continuous arm manipulation, online improvement with limited real-robot experience, dexterous hands, and residual control. However, PPO can be easier to engineer and scale in very high-throughput GPU simulation. Neither algorithm is universally superior; comparisons require matched environments and interaction budgets.

3.3 PPO versus SAC

Property	PPO	SAC
Category	On-policy	Off-policy
Reuse of older data	Low	High
Replay buffer	Usually unnecessary	Required
Massive parallel simulation	Highly established	Possible, with more design work
High-dimensional continuous actions	Supported	Supported
Real-world sample efficiency	Often lower	Often potentially better
Implementation complexity	Relatively straightforward	Relatively higher
Legged locomotion	Very common	Less common
Arms and dexterous hands	Common	Also used / suitable
Recommended starting point	Simulation-based control baseline	Data-expensive continuous control

Practical recommendation: reproduce PPO first for simulation-based bipedal or quadrupedal control. Investigate SAC when limited physical interaction data and online adaptation are central. Use identical tasks, interaction counts, and evaluation protocols for comparisons.

Chapter 4: Designing a Motion-Control RL Environment

The core of a training environment is

$$\text{Environment} = \{\text{Observation}, \text{Action}, \text{Reward}, \text{Reset}, \text{Dynamics}\}.$$

Goal-conditioned tasks also require a Command or Goal definition. In practice, environment design can matter more than minor changes to a PPO network.

4.1 Observation: what can the robot sense?

Observation	Meaning	Typical source
Joint position	Joint angles or positions	Encoders
Joint velocity	Joint speeds	Encoders / estimators
Base angular velocity	Body rotational velocity	IMU
Projected gravity	Gravity direction expressed in body frame	Orientation estimator
Previous action	Previous control command	Controller
Velocity command	Requested robot velocity	High-level controller
Contact state	Foot / fingertip contact estimates	Contact estimator
Height map	Terrain elevation	Depth camera or other sensors

A representative proprioceptive observation is

$$o_t = [q_t, \dot{q}_t, \omega_t, g_t, a_{t-1}, c_t].$$

Always distinguish **states available in simulation** from **observations accessible on the deployed robot**. Base linear velocity, contacts, and exact object poses often require estimators on hardware.

Observation history

Motion control is frequently partially observable. Current joint angles alone do not reveal a previous slip, an external push, or actuator delay. One solution is to use a history window:

$$o_t^{\text{hist}} = [o_{t-k}, \dots, o_{t-1}, o_t].$$

An MLP, GRU, or Transformer can process this history and help the policy infer hidden dynamical and contact properties.

4.2 Action: what should RL output?

Action space	Output	Typical characteristics
Joint position	Target joint positions	A stable and accessible baseline
Joint velocity	Target joint velocities	Suitable for velocity-control interfaces
Joint torque	Target joint torques	Direct control; harder training and safety challenges
Cartesian action	End-effector pose or velocity	Often natural for robot arms

A. Joint-position control

A common parameterization is

$$q_t^{\text{target}} = q^{\text{default}} + s_a a_t, \qquad a_t = \pi_\theta(o_t).$$

A low-level PD controller then computes

$$\tau_t = K_p(q_t^{\text{target}} - q_t) - K_d \dot{q}_t.$$

For reference tracking, one can instead use

$$q_t^{\text{target}} = q_t^{\text{ref}} + s_a a_t.$$

This is a residual action around the reference. Using PD control relieves the learned policy from modeling every aspect of actuator control.

B. Joint-torque control

The policy directly predicts $\tau_t = \pi_\theta(o_t)$. This gives greater direct control authority, but current, torque, temperature, speed, and position constraints must be enforced explicitly.

C. Cartesian control

For a robot arm, an action may be $a_t = [\Delta x, \Delta y, \Delta z, \Delta R]$. An IK or OSC module maps the Cartesian command into joint commands.

```
RL Policy -> End-effector increment / target -> IK / OSC -> Joints -> Arm
```

The orientation increment ΔR can be represented using a rotation vector or another consistent parameterization. Isaac Lab offers several joint-space and Cartesian control interfaces [6].

Suggested starting points

Robot	Initial action-space recommendation
Quadruped / biped	Joint-position targets
Humanoid motion tracking	Joint-position targets / residuals
Conventional robot arm	Cartesian pose increments
Multi-fingered hand	Finger-joint position targets
Arm plus dexterous hand	Cartesian arm action plus finger-joint action

A policy's output need not match the final actuator interface. For example, the policy may predict a Cartesian pose change, while the motor drivers execute joint-position commands.

4.3 Reward: what behavior should the robot optimize?

A useful decomposition is

$$r_t = r_t^{\text{task}} + r_t^{\text{stability}} - r_t^{\text{regularization}}.$$

For bipedal velocity tracking, possible terms are

$$r_{\text{velocity}} = \exp\left(-\frac{\|v_t - v_t^{\text{cmd}}\|^2}{\sigma_v^2}\right),$$

$$r_{\text{orientation}} = \exp\left(-\frac{\|e_t^{\text{orientation}}\|^2}{\sigma_R^2}\right),$$

$$r_{\text{smooth}} = -\|a_t - a_{t-1}\|^2.$$

One combined reward is

$$r_t = w_v r_{\text{velocity}} + w_R r_{\text{orientation}} - w_a\|a_t - a_{t-1}\|^2 - w_\tau\|\tau_t\|^2.$$

Actual tasks may add body-height, foot-slip, collision, joint-limit, energy, and contact-stability terms. Rewarding only forward velocity may encourage lunging, implausibly fast leg motion, or exploitation of simulator artifacts. Conversely, overly strong regularization may cause the robot to remain still.

Principle: rewards should capture both task accomplishment and safe, stable, deployable behavior.

4.4 Reset: which initial states should be sampled?

Simple locomotion may start from a standing pose. Motion tracking and multistage manipulation should not always begin from one fixed state, or the policy may fail to recover when entering a task midway.

Uniform Reference State Initialization (Uniform RSI): sample a time from a reference motion,

$$t_0 \sim U(0, T),$$

then initialize the robot near the corresponding reference state.

Event-biased RSI: increase the sampling probability near takeoff, landing, contact switches, grasp initiation, or release. Ensure sampled states are physically and contact consistent; unrealistic resets can make learning harder rather than easier.

Chapter 5: Training a Complete Motion Controller

5.1 The standard workflow

```
Robot model (URDF / MJCF / USD)
              |
              v
Physics simulator (Isaac Lab / MuJoCo)
              |
              v
Define Observation / Action / Reward / Reset
              |
              v
         PPO / SAC training
              |
              v
       Simulation evaluation
              |
              v
Domain Randomization / Curriculum
              |
              v
         Sim-to-sim test
              |
              v
         Sim-to-real transfer
              |
              v
              Real robot
```

5.2 Step 1: build a robot simulation model

Component	What must be modeled
Robot model	Link masses, inertias, joint axes, joint limits
Actuator	Motor torque and speed limits, friction, response
Contact	Friction, collision geometry, solver settings

Component	What must be modeled
Sensors	Encoders, IMUs, visual sensors
Control	PD gains, update frequencies, action delay
Environment	Ground, obstacles, manipulated objects

A URDF is not merely a visualization asset: it contains important dynamical information. However, the presence of a parameter in a file does not imply that it has been calibrated against the real robot.

5.3 Step 2: choose control frequencies

The following values are illustrative rather than hardware-specific:

Component	Example frequency
Physics simulation	500 Hz
Joint PD control	500 Hz
RL policy	50 Hz
Vision update	30 Hz
High-level planning	5 Hz

If $f_{\text{sim}} = 500\,\text{Hz}$ and $f_{\text{RL}} = 50\,\text{Hz}$, the simulator advances ten physics steps per policy action. This is called **action decimation**. The high-frequency control loop executes actions while RL updates its goals less often.

5.4 Step 3: parallel training

An **illustrative** PPO configuration for legged locomotion is

Parameter	Example setting
Parallel environments	4096
RL frequency	50 Hz
Rollout length	24 steps per environment
Algorithm	PPO
Actor / Critic	MLP / MLP
Action	Joint-position target
Observation	Proprioception + command

One rollout gathers $4096 \times 24 = 98{,}304$ environment transitions. This counts interaction samples, not gradient updates. Visual control or contact-heavy dexterous manipulation may require fewer parallel environments.

The actor–critic structure is

```
Observation
   |
   +-------> Actor  -----> Action -----> Simulator
   |                                       |
   +-------> Critic -----> Value <------- Reward
```

Both actor and critic are updated during training; typical deployment runs the actor only.

Chapter 6: Motion-Imitation Reinforcement Learning

Direct exploration can be difficult for jumping, dance, gymnastics, and high-dimensional dexterous motions. Reference motions provide useful guidance.

6.1 The basic pipeline

Motion Tracking / Motion Imitation RL learns physical tracking behavior with environment rewards and rollouts; it is not equivalent to supervised action-label BC. For BC/ACT/generative action-chunk training, consult Data and Imitation Learning.

```
Human motion / Teleoperation
             |
             v
     Motion reconstruction
             |
             v
      Motion retargeting
             |
             v
     Robot reference motion
             |
             v
      Motion-tracking RL
             |
             v
       Robust motion policy
```

Reference data may originate from motion capture, human-video reconstruction, teleoperation, existing controllers, or motion planners.

6.2 Motion-tracking rewards

For reference joint positions q_t^{ref}, a simple reward is

$$r_{\mathrm{track}} = \exp\left(-\frac{\|q_t - q_t^{\mathrm{ref}}\|^2}{\sigma_q^2}\right).$$

Whole-body tasks may also track root position, root orientation, joint velocity, end-effector positions, and contact schedules. Similar joint angles alone do not guarantee a physically stable movement. The policy must balance **motion fidelity, dynamic feasibility, and stability**.

6.3 DeepMimic

DeepMimic is a classic example of combining reference-motion imitation rewards with task rewards to train physical characters to perform complex actions [2].

```
Reference motion -> Tracking reward --+
                                      +--> RL policy -> Physics
Task specification -> Task reward ----+
```

Project: https://xbpeng.github.io/projects/DeepMimic/

6.4 AMP: Adversarial Motion Priors

Conventional motion tracking follows a specified reference at each time step. Adversarial Motion Priors (AMP) instead train a discriminator to provide a motion-style reward from reference datasets. A policy can then accomplish its task while exhibiting motions consistent with that prior, such as a natural-looking gait, without strictly reproducing one reference frame by frame [3].

6.5 Motion tracking versus AMP

Property	Motion-tracking RL	AMP
Requires motion data	Typically reference trajectories	Motion examples
Explicitly follows a specified trajectory	Yes	Not necessarily
Reward source	Reference tracking error	Learned motion-prior reward
Primary objective	Reproduce a target motion	Match a style / motion distribution
Major challenge	Temporal alignment and stability	Discriminator training and reward balancing

6.6 From tracking one clip to universal control

Projects such as HOVER investigate unifying multiple motion-command modes within a whole-body policy so that the controller is not restricted to reproducing a fixed sequence [4].

Trend: single-motion tracking policy → multimode / universal motion controller.

Chapter 7: Teacher–Student Reinforcement Learning

7.1 Why use teacher–student training?

A simulator can expose exact object positions and velocities, contact forces, friction coefficients, and hidden dynamical parameters. These may be unavailable or noisy in a real robot. A policy that directly depends on them may fail to transfer.

```
Full simulation state:
object poses, contact forces, friction, dynamics
                    |
```

```
                    v
                Teacher policy
                    |
                    v
                Teacher actions
                    |
                 Distillation
                    |
                    v
Real observations (Camera / Encoder / IMU) -> Student policy
```

7.2 Standard training procedure

First train a teacher with privileged state information:

$$a_t^T = \pi_T(s_t).$$

Then train a student with deployable observations only:

$$a_t^S = \pi_S(o_t).$$

A common action-distillation loss is

$$\mathcal{L}_{\text{distill}} = \mathbb{E}\left[\|a_t^S - a_t^T\|^2\right].$$

The student may use observation histories and learned state estimators to infer hidden properties. Practical distillation also needs to account for differences between the state distributions visited by teacher and student.

7.3 Asymmetric actor–critic versus teacher–student

Asymmetric actor–critic: the actor uses only deployable observations from the start of training, while the critic can use privileged simulation information.

Teacher–student: a teacher actor may itself use privileged information, and its behavior is subsequently transferred to a student actor with limited observations.

The first is a **training architecture**; the second is a **policy-transfer procedure**. They can be combined. Isaac Lab and RSL-RL support relevant observation groups and training configurations [5] [6].

7.4 RMA: Rapid Motor Adaptation

Rapid Motor Adaptation (RMA) integrates a motor policy with a mechanism for adapting to unknown dynamics. Recent observation history provides information that helps the robot respond to changing terrain, friction, payload, and dynamics [7].

Paper: https://arxiv.org/abs/2107.04034

Chapter 8: Case Study I — Training Unitree G1 Walking with PPO

8.1 Task definition

Train the G1 to move stably under $c_t = [v_x^{\text{cmd}}, v_y^{\text{cmd}}, \omega_z^{\text{cmd}}]$, including forward/backward movement, lateral movement, and turning.

8.2 Observation

Start with proprioception: joint positions, joint velocities, base angular velocity, projected gravity, previous actions, and velocity commands. For stairs or uneven terrain, gradually add height scans, contact estimates, observation histories, and visual features.

8.3 Action

Use joint-position targets:

$$q_t^{\text{target}} = q^{\text{default}} + s_a \pi_\theta(o_t),$$

executed by a low-level PD controller. This makes it easier to reuse existing hardware interfaces.

8.4 Reward components

Reward	Purpose
Linear velocity tracking	Track longitudinal and lateral velocity
Angular velocity tracking	Track yaw rate
Orientation stability	Maintain appropriate body posture
Action smoothness	Reduce oscillations
Joint-limit penalty	Avoid extreme joint configurations
Foot-slip penalty	Reduce foot sliding
Termination penalty	Penalize falls and failures

8.5 Curriculum learning

Stage	Training condition	Goal
1	Standing	Basic balance
2	Slow forward walking	Basic gait
3	Forward/backward/lateral walking	Broader speed range
4	Turning	Angular-velocity tracking
5	External pushes	Robustness and recovery
6	Uneven terrain	Contact adaptation
7	Stair climbing	Challenging locomotion

The command and terrain distributions can be expanded continuously. A separate controller need not be retrained from scratch for every stage.

8.6 Open-source starting point

Unitree RL Lab builds on Isaac Lab and includes training and deployment-related code for Unitree platforms [8]. The following commands are **illustrative for the repository interface**; verify the current tasks and dependencies in the checked-out version:

```
# List registered tasks
./unitree_rl_lab.sh -l

# Train G1 velocity tracking (illustrative task name)
./unitree_rl_lab.sh -t --task Unitree-G1-29dof-Velocity

# Evaluate a trained policy
./unitree_rl_lab.sh -p --task Unitree-G1-29dof-Velocity
```

First controlled study: reproduce flat-ground walking, then ablate action smoothness, domain randomization, observation history, and the foot-slip penalty. Compare convergence, velocity tracking, and perturbation recovery.

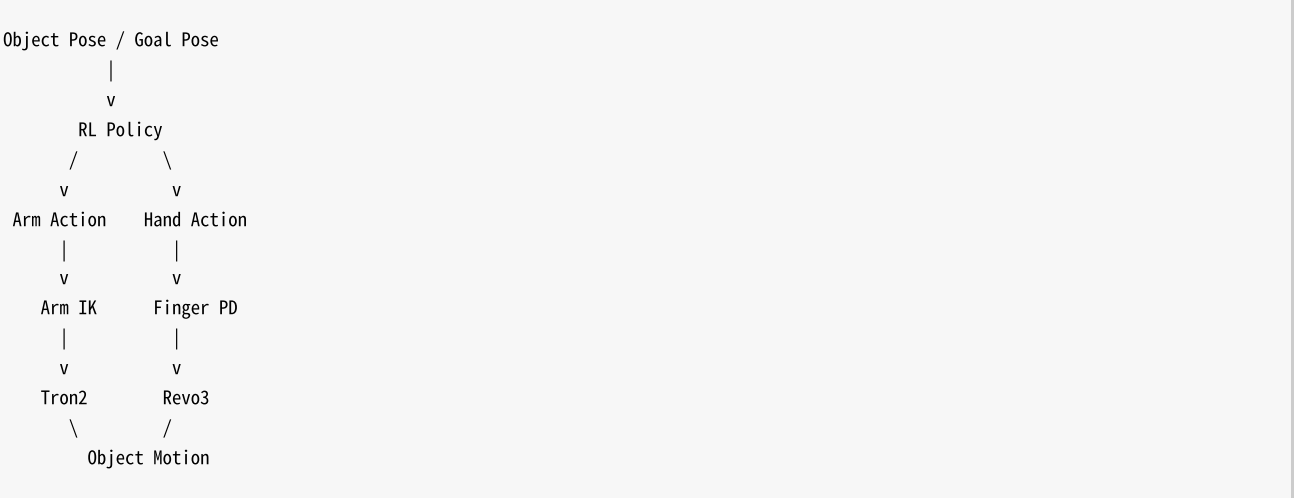
Chapter 9: Case Study II — Arm and Dexterous-Hand RL

Consider a **Tron2 arm, Revo3 dexterous hand, and screwdriver** to illustrate how locomotion-style RL environment design extends to contact-rich manipulation.

9.1 Task definition

1. Pick up a screwdriver from multiple positions on a tabletop.
2. Coordinate arm posture and finger contacts.
3. Move the screwdriver to a designated location.
4. Rotate it by a desired angle.
5. Maintain or recover the grasp after disturbances.

9.2 Recommended control architecture



```
            |
            v
      Observation Feedback
```

It is reasonable to train independent arm and hand controllers first, then move toward a coupled policy.

9.3 Three training routes

Route A: RL from scratch. Define rewards for grasping, lifting, rotation, and success:

$$r_t = r_{\text{grasp}} + r_{\text{lift}} + r_{\text{rotate}} + r_{\text{success}}.$$

This does not require reference trajectories, but exploration in a high-dimensional contact problem is difficult.

Route B: motion-tracking RL. Reconstruct reference end-effector and finger trajectories, $T^{\text{ref}}_{\text{hand}}(t)$ and $q^{\text{ref}}_{\text{finger}}(t)$, from human videos or teleoperation. This reduces exploration difficulty, but one demonstration rarely covers all initial object poses and contact variations.

Route C: residual RL. Add a constrained correction to a trajectory tracker, an IK controller, or an existing policy:

$$a_t = a_t^{\text{base}} + \Delta a_t^{\text{RL}}.$$

For example, the baseline could track an arm-end-effector trajectory while RL adjusts finger contacts; alternatively, RL can produce limited corrections for both arm and hand. Residual magnitudes, action rates, and contact forces must be constrained.

9.4 Observation design

One possible observation vector is

$$o_t = [q_{\text{arm}}, q_{\text{hand}}, \dot{q}, \hat{T}_{\text{object}}, T_{\text{goal}}, a_{t-1}].$$

Here $\hat{T}_{\text{object}}$ is estimated from head- or wrist-mounted cameras. If object velocity is not directly observable, histories or a state estimator can help infer it. **Do not silently give the actor ground-truth object state that only exists in simulation.** The critic, however, may use privileged information.

9.5 Reward design

Reward	Role
Reach	Move the hand near the target
Grasp	Establish a stable grasp
Lift	Lift the object
Position	Move the object to the target region
Rotation	Achieve the desired rotation

Reward	Role
Stability	Reduce slip and drops
Smoothness	Avoid action oscillations
Success	Reward full task completion

A policy with only a terminal rotation-success reward may rarely discover useful grasps through random exploration. A curriculum and carefully chosen intermediate rewards often help.

9.6 Proposed training stages

Stage	Task	Goal
1	Maintain a pre-established grasp	Basic contact control
2	Rotate an already-held object	In-hand manipulation
3	Reach and grasp	Grasp acquisition
4	Grasp, then rotate	Multistage operation
5	Randomize initial object pose	Generalization
6	Add object-pose estimation errors	Visual robustness
7	Add external disturbances	Recovery
8	Evaluate real-robot closed-loop control	Test transfer

When reference trajectories are available, Uniform RSI and Event-biased RSI can emphasize critical segments such as contact establishment, release, and object rotation.

Chapter 10: Simulation-to-Real Transfer

10.1 Why simulation success does not ensure hardware success

Mismatch	Typical cause
Mass / inertia	Errors in link mass and inertia
Friction	Incorrect ground or object friction
Actuator	Motor response, dead zones, internal controllers
Delay	Communication and control latency
Observation noise	Encoder, IMU, and visual error
Contact	Contact solver approximations
Geometry	Incorrect collision geometry
Calibration	Joint zero positions, frame transforms, extrinsics

For instance, a simulated motor may respond much faster than its physical counterpart; the resulting timing error can induce oscillations.

10.2 Domain randomization

Randomize environment and dynamics parameters during training:

$$\xi \sim p(\xi).$$

The vector ξ may include link mass, center of mass, friction, motor strength, action delay, and sensor noise.

Randomized parameter	Purpose
Mass	Robustness to mass mismatch
Center of mass	Robustness to COM errors
Friction	Robustness to contact variation
Motor strength	Robustness to actuator variation
Action delay	Robustness to latency
Observation noise	Robustness to sensor errors
External pushes	Robustness to disturbances

Ranges should reflect calibration, hardware specifications, and physically plausible uncertainty. Wider randomization is not automatically better.

10.3 Sim-to-sim evaluation

A policy trained in Isaac Lab can be tested in MuJoCo, or vice versa, to detect over-reliance on simulator-specific numerical behavior.

```
Isaac Lab Training -> Export Policy -> MuJoCo Evaluation
                                            |
                                            v
                                    Real Robot Test
```

If the controller behaves very differently between engines, inspect dynamics, contact, and action interfaces first. Passing sim-to-sim tests is additional evidence, not a guarantee of real-world safety.

10.4 Real-robot deployment checklist

- Match observation dimensions, ordering, and normalization to training.
- Verify joint indexing, sign conventions, units, coordinate frames, and zero positions.
- Validate action scales, PD gains, control rates, and execution timing.
- Enforce position, velocity, torque, and temperature limits.
- Provide safe fallback behavior, communication watchdogs, and emergency stops.

- Use fall protection for bipeds; guard against excessive contact forces, jamming, and dropped objects for dexterous hands.
- Begin with low-risk, small-amplitude tests before expanding the task envelope.

Advanced topic: learning the simulation–reality dynamics gap

Work such as ASAP uses data from real robots to help align simulated and physical dynamics, thereby improving sim-to-real transfer of agile whole-body behavior [9].

Chapter 11: Evaluating Motion-Control RL

An increasing training reward does not necessarily imply a better deployable controller. Use held-out conditions and independent task metrics.

11.1 Legged-robot metrics

Metric	Meaning
Success rate	Fraction of completed tasks
Fall rate	Frequency of falls
Velocity tracking error	Difference between actual and commanded velocity
Orientation error	Body pose deviation
Energy consumption	Power or energy cost
Foot slip	Unwanted foot sliding
Recovery rate	Success after perturbations
Generalization	Performance on unseen terrain or parameters

11.2 Motion-tracking metrics

Metric	Meaning
Joint tracking error	Joint trajectory deviation
Root position error	Base-position deviation
Root orientation error	Base-orientation deviation
End-effector error	End-effector tracking deviation
Contact consistency	Agreement with intended contacts
Motion completion rate	Fraction of completed motion clips
Recovery rate	Robustness after disturbances

11.3 Dexterous-manipulation metrics

Metric	Meaning
Task success rate	Fraction of successful manipulation trials
Object pose error	Final or trajectory pose deviation
Grasp stability	Ability to retain a stable grasp
Object drop rate	Frequency of dropped objects
Contact slip	Relative slip at contacts
Recovery rate	Recovery after disturbance
Completion time	Time required to finish a task
Generalization	Performance under unseen object poses

Fair comparison: match the environment and task, the interaction budget, and test conditions; report multiple random seeds, unseen initial states, and metrics independent of the training reward. Keep training success, simulated generalization, and real-world performance separate.

Chapter 12: Open-Source Frameworks and Tools

Framework	Main use	Recommendation
Isaac Lab	General robotic simulation and RL	High priority
RSL-RL	PPO, motion control, distillation	High priority
Unitree RL Lab	Unitree robot training and deployment	Useful for G1/H1
MuJoCo Playground	GPU-accelerated robot learning	Worth comparing
legged_gym	Classic legged PPO codebase	Useful for foundations
skrl	Multiple RL algorithms	Useful for comparisons

12.1 Isaac Lab

Repository: https://github.com/isaac-sim/IsaacLab

Isaac Lab covers legged locomotion, humanoid locomotion, manipulation, dexterous manipulation, and contact-rich tasks. Examples of environment IDs from an official versioned catalog include [6]:

Example environment	Task
`Isaac-Velocity-Flat-G1-v0`	G1 flat-ground velocity tracking
`Isaac-Velocity-Rough-G1-v0`	G1 rough-terrain velocity tracking
`Isaac-Velocity-Flat-Unitree-Go2-v0`	Go2 flat-ground locomotion
`Isaac-Reach-Franka-v0`	Franka reaching

Example environment	Task
`Isaac-Lift-Cube-Franka-v0`	Franka cube lifting
`Isaac-Repose-Cube-Allegro-v0`	Allegro hand object reorientation

IDs may change between releases, so consult the task registry of the installed version.

12.2 RSL-RL

Repository: https://github.com/leggedrobotics/rsl_rl

A productive code-reading sequence is **Runner** → **PPO** → **Actor/Critic** → **Rollout Storage** → **Distillation**. For PPO research in motion control, this is a practical baseline implementation [5].

12.3 MuJoCo Playground

Repository: https://github.com/google-deepmind/mujoco_playground

MuJoCo Playground provides GPU-accelerated learning tasks for quadrupeds, humanoids, dexterous hands, and robot arms. Some batched dynamics tasks can be more lightweight than running a full rendering-focused simulator, but performance depends on GPU, contact complexity, visual input, and the simulation backend [10].

12.4 Which framework should you choose?

Research goal	Suggested starting point
Learn PPO for motion control	Isaac Lab + RSL-RL
Train a G1 walking controller	Unitree RL Lab
Massive GPU-parallel RL	Isaac Lab / MuJoCo Playground
Dexterous manipulation	Isaac Lab / MuJoCo Playground
Combine vision and control	Isaac Lab
Modify PPO itself	RSL-RL
Compare SAC and PPO	An RL stack supporting both methods
Lightweight control experiments	MuJoCo Playground

Chapter 13: Representative Papers and Reading Order

Paper / project	Year	Core topic	What to learn
DeepMimic	2018	Motion-imitation RL	Reference rewards and RSI
AMP	2021	Adversarial motion priors	Style rewards from motion datasets
RMA	2021	Rapid adaptation	Observation history and dynamics

Paper / project	Year	Core topic	What to learn
Learning to Walk in Minutes	2022	Parallel PPO	Large-scale locomotion training
Learning Robust Perceptive Locomotion	2022	Perceptive locomotion	Terrain perception and transfer
HOVER	2025	Multimode whole-body control	Universal command representations
ASAP	2025	Dynamics-gap alignment	Sim-to-real
BeyondMimic	2025	Complex humanoid motions	Tracking and skill generalization
MuJoCo Playground	2025	GPU robot learning	Modern experimentation platform

Four particularly useful starting points are:

1. **Learning to Walk in Minutes:** parallel PPO, reward design, curriculum learning, and sim-to-real [1].
2. **DeepMimic:** reference motion, imitation rewards, reference-state initialization, and physics-based motion tracking [2].
3. **RMA:** privileged information, observation history, motor adaptation, and deployment [7].
4. **HOVER:** whole-body control, multiple command modalities, tracking, and distillation [4].

Chapter 14: Learning and Research Roadmap

Stage 1: basic motion-control RL

Goal: train a quadruped or biped to walk stably.

Learn Isaac Lab, RSL-RL, PPO, observation/action/reward design, and domain randomization. First reproduce velocity tracking on a platform such as G1; then ablate reward terms and observations.

Stage 2: motion-tracking RL

Goal: track a reference while maintaining dynamic feasibility and recovery ability.

Study motion retargeting, tracking rewards, RSI, curricula, and residual RL. Start from one reference motion, then vary initial states and external disturbances.

Stage 3: teacher–student and sim-to-real

Goal: eliminate dependence on simulator-only ground-truth state.

Study asymmetric actor–critic, privileged information, distillation, observation histories, and domain randomization. Train a privileged teacher, transfer its behavior to a deployable student, and test the controller in another simulator.

Stage 4: complex motion control

Direction	Example tasks
Humanoid locomotion	Stairs, obstacles, jumping, fall recovery
Dexterous manipulation	In-hand rotation, tool use, disturbance recovery

Focus on contact dynamics, task curricula, strong robustness, and closed-loop hardware control.

Stage 5: universal motion controllers

Investigate a conditional policy that can handle multiple commands:

$$a_t = \pi_\theta(o_t, c_t).$$

The condition c_t may encode desired velocity, target pose, a reference-motion segment, desired object motion, or skill identity. Related directions include goal-conditioned motion control, universal motion tracking, hierarchical motion policies, residual motor adaptation, and learning-based whole-body control.

Chapter 15: Summary and Main Takeaways

Motion-control RL is best understood along three dimensions: **task category, training strategy, and engineering workflow**.

Task	Central question
Locomotion	How can a robot move stably?
Agile locomotion	How can it execute dynamic maneuvers?
Motion tracking	How can it reproduce reference motions accurately?
Manipulation	How can it manipulate objects?
Dexterous manipulation	How can it coordinate multiple finger contacts?
Loco-manipulation	How can it coordinate movement and manipulation?

Method	Main question addressed
PPO	How can large parallel simulation train controllers?
SAC	How can interaction data be reused efficiently?
Imitation RL	How can reference demonstrations ease exploration?
Residual RL	How can an existing controller be improved?
Teacher–student	How can a privileged policy be transferred to real observations?
Adaptive RL	How can the controller handle unknown dynamics and environments?

The complete development workflow is

$$\boxed{\begin{array}{c}\text{Robot Modeling} \rightarrow \text{Task Design} \rightarrow \text{PPO/SAC Training} \\ \text{Curriculum} + \text{DR} \rightarrow \text{Sim-to-Sim} \rightarrow \text{Sim-to-Real}\end{array}}.$$

Takeaway 1: task design matters as much as the algorithm. Observations, actions, rewards, resets, simulation fidelity, and curriculum schedules strongly influence results.

Takeaway 2: PPO plus parallel simulation remains an important baseline. It is well established in legged locomotion and motion tracking. When real interaction data are scarce, algorithms with greater data reuse, such as SAC, are worth considering.

Takeaway 3: general-purpose, conditioned, robust motion controllers are a promising direction. One coherent research progression is

Motion Tracking RL → Residual RL → Robust Sim-to-Real → Goal-conditioned Universal Motion Control.

References and Open-Source Links

The following links point primarily to papers, project pages, and official repositories. Confirm final bibliographic details and currently supported task IDs from the relevant publication and software release.

1. Rudin et al., *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning* (2022)
2. Peng et al., *DeepMimic* (2018)
3. Peng et al., *AMP: Adversarial Motion Priors* (2021)
4. HOVER: Versatile Neural Whole-Body Controller for Humanoid Robots; paper
5. RSL-RL official repository
6. Isaac Lab official repository; environment catalog
7. RMA: Rapid Motor Adaptation for Legged Robots
8. Unitree RL Lab official repository
9. ASAP: Aligning Simulation and Real-World Physics
10. MuJoCo Playground official repository
11. legged_gym official repository
12. skrl official repository

Document version: October 15, 2026. Numerical settings are instructional examples, not validated safety configurations for real robots.